

Imagine developing an e-commerce platform with the following architecture:

- Microservices Architecture:

- Services: User Management, Product catalog, Shopping Cart, Order Processing, Payment and Notification.
- Benefits: Each service can be developed, deployed and scaled independently, enhancing flexibility and resilience.

- Event-Driven Architecture:

- Mechanism: Services communicate through events (e.g., "Order Placed", "Payment Processed").
- Benefits: Promotes loose coupling and allows services to react to events asynchronously, improving scalability and responsiveness.

Applying SOLID Principles

- Single Responsibility Principle: Each microservice handles a specific business function.
- Open/Closed Principle: Services can be

extended with new features without modifying existing code.

→ Liskov Substitution Principle:
Interchangeable service implementation without affecting the system's behavior.

→ Interface Segregation Principle:
Services expose only necessary interfaces, preventing clients from depending on unused functionalities.

→ Dependency Inversion Principle:
High-level modules depend on abstractions, allowing for flexible and testable code.

Observing DRY and FISS Principles

→ DRY (Don't Repeat Yourself):

- Utilize shared libraries for common functionalities like logging and authentication.
- Centralize configuration management to avoid redundancy.

→ KISS (Keep it Simple, Stupid):

- Design Services with clean and concise responsibilities.
- Avoid over-engineering; implement features as needed.